Le board se met à jour tout seul

git → Linear, avec n8n en local



Parfait Ahouanto

LE PROBLÈME

Le board ment

> **« Ce board reflète-t-il ce sur quoi on travaille en ce moment ? »**

Poser la question. Laisser le silence répondre.

3 tickets en Todo

0 ligne de code restant à écrire

LA VRAIE RAISON

Ce n’est pas de la négligence

« Déplacer une carte n’apporte rien d’immédiat à celui qui la déplace. »

Le développeur commence une tâche. Le ticket, lui, ne bouge pas. Parfois il n’existe même pas encore. Le chef de projet n’a alors aucune idée de ce qui se passe réellement.

QUI PAIE

le développeur

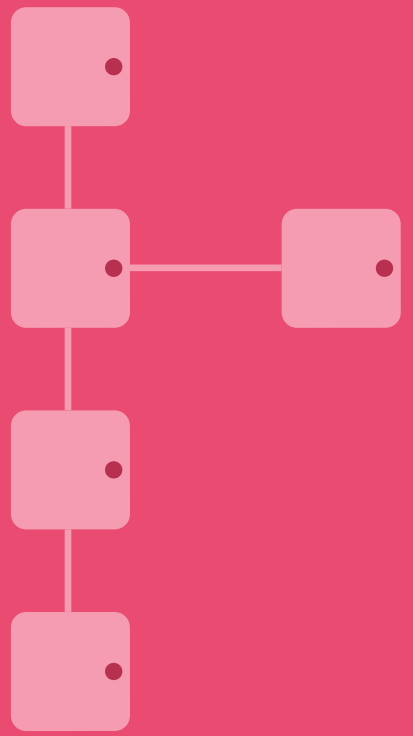
QUI EN PROFITE

le chef de projet

Suivre l’avancement, évaluer la charge de chacun, savoir qui est disponible : utile, mais jamais pour celui qui déplace la carte.

Le board se met à jour
à partir de la seule chose
qu'un développeur fait toujours

commiter



01

DÉMO

Le cas simple

L'identifiant du ticket est déjà dans le nom de la branche.

feat/ALO-42-relance-paiement → ALO-42 passe en In Progress

Aucune IA ici

n8n lit la branche, trouve la référence, met le ticket à jour. Instantané et gratuit.
Montrer que la base fonctionne sans magie rend la suite crédible.

02

DÉMO

Le suivi continu

Plusieurs commits sur le même ticket, sans jamais le fermer soi-même.

git-sync → chaque commit ajoute un commentaire, le ticket reste In Progress

Rien ne se perd

Chaque synchro ajoute son propre commentaire d'historique. Le ticket ne bouge que lorsque le développeur le décide explicitement.

03

DÉMO

Le passage en Done

Le mot-clé closes-all, et de vrais tests, jamais une estimation.

git commit -m "... (closes-all)" → tests réels exécutés en local → Done

Une vérification réelle, pas une supposition

Les tests tournent sur la machine du développeur, avant tout envoi. n8n ne lit qu'un résultat pass/fail, jamais de code, jamais de devinette.

AVANT L'ARCHITECTURE

n8n, en deux mots

Un workflow, c'est une suite de nœuds reliés par des flèches, qui s'exécute automatiquement à chaque déclenchement. Un outil visuel, plutôt qu'un programme classique.



Déclencheur

démarre le workflow (ici, un webhook appelé par git-sync)

Logique (IF)

teste une condition, choisit une branche : vrai ou faux

Code

transforme ou calcule une donnée avec un peu de JavaScript

Action (API)

appelle un service externe : Linear, Claude

AVANT L'ARCHITECTURE

Comment un workflow s'exécute

Dans l'ordre des flèches

Chaque nœud reçoit la sortie du précédent, et s'exécute dans l'ordre des connexions dessinées sur le canvas.

Les nœuds IF créent des branches

Un workflow peut suivre un chemin ou un autre selon une condition : vrai d'un côté, faux de l'autre, jamais les deux à la fois.

Chaque passage est une Execution

Visible dans l'onglet Executions de n8n, nœud par nœud, avec les données exactes à chaque étape : l'outil de diagnostic principal.

POUR ALLER PLUS LOIN

Architecture interne de n8n

Techniquement, n8n est une application Node.js / TypeScript, structurée en plusieurs briques.

Moteur d'exécution

(n8n-core)

Sait lire un workflow, le JSON qu'on édite directement, et le faire tourner.

Les nœuds

(n8n-nodes-base)

Chaque type (Webhook, Code, IF, HTTP Request) est un module avec une fonction execute().

Serveur HTTP

(Express)

Sert l'éditeur visuel, expose l'API, écoute les webhooks entrants comme celui de git-sync.

Le JSON que tu édites, c'est le workflow

Pas de couche cachée : ce qu'on modifie dans workflows/git-linear-sync.json est exactement ce que le moteur execute.

POUR ALLER PLUS LOIN

Le format JSON d’un workflow

Pas de couche cachée : ce qu’on édite dans workflows/git-linear-sync.json est exactement ce que le moteur exécute.

```
{
  "nodes": [
    { "name": "Webhook", "type": "n8n-nodes-base.webhook",
      "parameters": { "httpMethod": "POST", "path": "git-sync" } },
    { "name": "Analyser le contexte", "type": "n8n-nodes-base.code",
      "parameters": { "jsCode": "..." } }
  ],
  "connections": {
    "Webhook": { "main": [[{ "node": "Analyser le contexte" }]] }
  }
}
```

nodes

tableau, un objet par nœud sur le canvas

name

le nom affiché, utilisé par connections

type

le genre de nœud : webhook, code, if…

parameters

la config propre à ce nœud

connections

les flèches entre les nœuds

POUR ALLER PLUS LOIN

Du webhook à la réponse, étape par étape

1	**Réception**	le serveur HTTP reçoit le POST de git-sync sur /webhook/git-sync
2	**Association**	il retrouve le workflow lié à ce chemin, démarre une exécution
3	**Parcours du graphe**	le moteur suit les connections, nœud par nœud
4	**Format commun**	chaque nœud reçoit et renvoie un tableau [{ json: {…} }]
5	**Branches IF**	sortie 0 = vrai, sortie 1 = faux : détermine la suite
6	**Réponse**	le nœud Respond to Webhook renvoie la réponse HTTP
7	**Traçabilité**	tout est sauvegardé, visible dans l'onglet Executions

POUR ALLER PLUS LOIN

n8n et Postgres

n8n utilise un ORM (TypeORM), une bibliothèque qui traduit du code en requêtes SQL, pour parler à Postgres.

DNS interne Docker : "postgres"

n8n
conteneur Docker

postgres
conteneur Docker

- docker-compose.yml fixe DB_POSTGRESDB_HOST=postgres, DB_POSTGRESDB_PORT=5432, etc.
- « postgres » n'est pas une adresse réseau classique : c'est le nom du service Docker, résolu automatiquement.
- Les deux conteneurs se parlent directement sur le réseau Docker, jamais via localhost ni la machine hôte.
- Chaque action (sauvegarder un workflow, enregistrer une exécution, lire une credential) devient une requête SQL.

POUR ALLER PLUS LOIN

Les états d’un nœud

AVANT L’EXÉCUTION

Actif

fait partie du workflow, s’exécute normalement à chaque passage

Désactivé

ignoré par le moteur : ses données d’entrée passent telles quelles au nœud suivant, sans exécuter son code

APRÈS UNE EXÉCUTION (ONGLET EXECUTIONS)

Succès

le nœud a tourné et renvoyé ses données

Erreur

le nœud a planté, l’exécution s’arrête là

Non atteint

branche IF non prise, ou nœud désactivé

Le piège vécu en direct

Claude · Rapprochement désactivé → Décider reçoit des données qu’il n’attend pas → erreur. Un nœud désactivé en amont peut casser un nœud en aval.

POUR ALLER PLUS LOIN

Les nœuds du workflow (1/2)

1	**Webhook**	reçoit l'appel POST de git-sync, déclenche l'exécution
2	**Analyser le contexte**	extrait branche/commits, cherche un identifiant PAR-X
3	**Référence trouvée ?**	IF : un identifiant de ticket a-t-il été trouvé ?
4	**Linear · Ticket par référence**	récupère le ticket par son numéro exact
5	**Linear · Contexte équipe**	liste les tickets ouverts de l'équipe
6	**Préparer l'appel IA**	construit le prompt à envoyer à Claude
7	**Claude · Rapprochement**	demande à Claude de deviner le bon ticket
8	**Décider**	lit la réponse de Claude, applique le seuil 0,7
9	**Correspondance sûre ?**	IF : la confiance dépasse-t-elle le seuil ?
10	**Linear · Créer le ticket**	crée un ticket si rien ne correspond
11	**Normaliser le ticket**	unifie les 3 chemins en un seul format

POUR ALLER PLUS LOIN

Les nœuds du workflow (2/2)

12	**Linear · État In Progress**	récupère l'identifiant de l'état In Progress
13	**Linear · Passer en In Progress**	change le statut du ticket
14	**Linear · Ajouter le commentaire**	poste le commentaire listant les commits
15	**Réponse**	renvoie la réponse HTTP à git-sync
16	**Mot-clé Done présent ?**	IF : closes-all est-il dans un commit ?
17	**Décider Done (tests réels)**	lit testResults, décide si tout est passé
18	**Tous les tests passent ?**	IF : le résultat des tests est-il au complet ?
19	**Linear · État Done**	récupère l'identifiant de l'état Done
20	**Linear · Passer en Done**	change le statut du ticket en Done
21	**Linear · Commentaire Done**	poste le récapitulatif des tests

CE QUI TOURNE DERRIÈRE

Les acteurs, de bout en bout

Développeur

Ecrit le code, commite (+ closes-all si le ticket est termine)

git-sync

Lit les commits, lance les tests en local si besoin, envoie tout a n8n

n8n

Reçoit l'appel et exécute les 21 nœuds pour décider quoi faire

Claude

Appelé seulement si aucune référence trouvée dans la branche

Linear

Reçoit la décision : statut + commentaire sur le ticket

Stockage en mémoire

Livres et auteurs, reinitialise au redemarrage (pas de base de donnees)

Tests d'acceptation

npm test, execute en local avant l'envoi (closes-all) resultat reel, pas une estimation

Postgres

Etat interne de n8n : workflows, executions, credentials chiffrees

Tout est local, sauf deux appels

n8n et Postgres tournent en Docker sur la machine, le script appelle localhost. Seuls Linear et Anthropic sont des appels sortants. Le passage en Done s'appuie sur de vrais tests exécutés en local, jamais sur une estimation de Claude, qui ne voit de toute façon jamais le code.

LE COÛT

Ce que ça coûte

21

nœuds

0

ligne déployée

1

soirée de travail

Abonnement

hébergement sur serveur

La licence

n8n est en fair-code. L'auto-hébergement est gratuit et illimité pour un usage interne ; seule la revente en tant que service est interdite.

CE QUI RESTE À DÉCIDER

On le met sur un serveur, ou il reste sur mon portable ?

- **Un abonnement** — un VPS, configuration de sécurité documentée
- **Export versionné** — le workflow vit dans le dépôt, pas dans une machine
- **Validation humaine** — à activer si la création automatique de tickets gêne

Les questions maintenant, les détails techniques après.

Merci

git → Linear, avec n8n en local

